Dated:

Coal Assignment 2

Mubiza Ali Asad
25k-0166
BAI-3B

Q1)
```
mov ax, 3
mob bx, ax
shl bx, 1
mov cx, ax
shl cx, 3
shl ax, 4
add ax, bx
add ax, cx
```
Answer = 004E

Q2)
```
.data
value Dword 2345ABCDh
.Code
mov eax, value
move ecx, 4
R: ror eax, 8
Loop R
mov ecx, 4
L: rol eax, 8
Loop L
```

Q3)
```
.data
value Dword 2345ABCDh
.Code
mov eax, value
mov ecx, 2
R: ror eax, 16
```

Dated:

```
Loop K
mov ecx, 2
L: rol eax, 11
Loop L
```

Q4 (-)

```
. data
  Bseconds Byte ?
. code
  mov dx, 0101100111011010
  mov ax, dx
  add ax, dx
  add al, 1Fh
  mov Bseconds, al
```

seconds = 11010 = 26

b)

```
. data
  Bminutes Byte ?
. code
  mov dx, 0101100111011010
  mov ax, dx
  shr ax, 5
  and ax, 3Fh
  mov Bminutes, al
```

minutes = 001110 = 14

D_x = 9A40h

A_x = 918Ah

CF = 1

Dated:

Q6 (a) test ax, OFFFFh
b) test bx, 1
c) test dx, 8000h
d) test dx, 8000h
e) test bl, OFFh

Q7) Dx = 0016h

Q8) 1) mov cx, 3 should be written as mov ecx, 8 because Jnp uses ecx in 32 bit code otherwise loop will work on garbage values.

2) mov esi, val and mov edi, val 2 load variables content instead of their address. Must make use of offset.

3) Dec esi and decode ; move backwards but the substraction must start at lowest byte and move afwards so they must be incremented.

4) The result is stored in [esi], which currently is Val 1. Must be stored in result.

Corrected code
Val 1 Q word 2040300436204 7A1h
Val 2 Q word 0552103047A2630B2h
result Q word 0
.code
mov ecx, 8

Dated:

```
mov. ebx, OFFSET result
mov, esi, OFFSET val 1
mov edi, OFFSET val 2
top:
 mov al, Byte Ptr [esi]
 sub al, Byte Ptr [edi]
  mov Byte Ptr [ebx], al
  inc esi
  inc edi;
  inc ebx
  loop top
```

Q a)

```
include irvine 32.inc
. code
main PROC
  call Read Dec
  cmp eax, 80
  jge grade A
  cmp eax, 60
  jge grade B
  cmp eax, 40
  jge grade C
  mov AL, 'F'
  jmp show
grade A: mov al, 'A'
jmp show
grade B : movadl 'B'
jmp show
grade C: mov al, 'C'
```

```
show:
call write char
exit
main ENDP
END MAIN
```

Dated:

Q10) EAX = 3, EBX = 10, empty stack

Q11)

```
mov eax, num1
cmp eax, num2
jle check-else-if
mov ebx, num3
cmp ebx, num4
je check
add eax, num3
mov result, eax
jmp done
check + else-if:
move eax, num2
cmp eax, num4
jne else-part
sub eax, num1
mov result
jmp done
else part:
mov result, 0
done
```

Q12)

```
while top:
mov eax, op2
cmp eax, op1
jg end-while
cmp eax, x
je else-part
cmp eax, y
jg else part
add 2,10
jmp after:if
else-part:
sub 2,10
after:if:
dec op2
jmp while top
end while
```